Hospital Management System - FastAPI & MySQL Project Documentation

Executive Summary

The Hospital Management System is a full-stack web application built with **FastAPI** (backend), **MySQL** (database), and **Jinja2 Templates** (frontend). This system enables hospital administrators and staff to manage doctor and patient records efficiently through an intuitive web interface. The application provides features for viewing, creating, and managing patient admissions and doctor information with real-time database synchronization.

Table of Contents

Project Overview

Purpose

The Hospital Management System streamlines hospital operations by providing a centralized platform to:

- **Manage Doctor Records**: Store doctor information including name, specialty, and contact details
- **Manage Patient Records**: Track patient admissions with comprehensive information (name, age, gender, contact, admission date, assigned doctor, and treatment status)
- **View Statistics**: Display real-time dashboard with patient and doctor counts
- **Patient Admission**: Add new patients to the system through an interactive web form

Key Objectives

✓ Create a responsive web interface for hospital staff
✓ Implement robust database operations with MySQL
✓ Provide seamless form submission and data validation
✓ Maintain data integrity with proper database relationships
✓ Enable quick access to critical hospital metrics

Technology Stack

Backend Framework

- **FastAPI** (v0.104+): Modern Python web framework for building APIs and web applications
    - Automatic OpenAPI documentation
    - Dependency injection system
    - Built-in form handling
    - Type hints and validation

Database

- **MySQL**: Relational database management system for persistent data storage
- **SQLModel**: ORM library combining SQLAlchemy and Pydantic for seamless database operations
    - Type-safe ORM
    - Automatic validation
    - Query optimization

Frontend

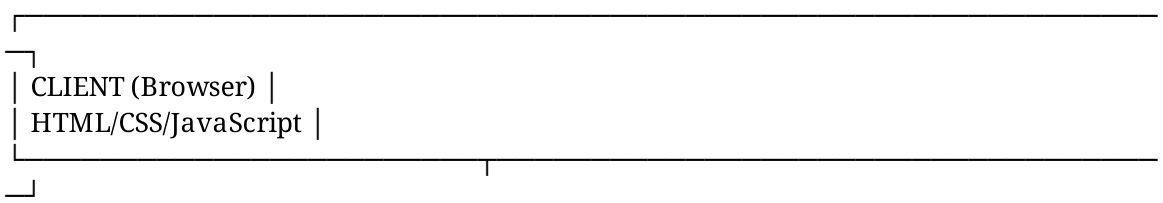
- **Jinja2 Templates**: Server-side templating engine for rendering HTML
- **HTML/CSS**: Markup and styling for user interface
- **Bootstrap/CSS**: Responsive design framework (used in static templates)

Development & Deployment

- **Uvicorn**: ASGI server for running FastAPI application
- **Python 3.9+**: Programming language
- **python-dotenv**: Environment variable management
- **mysql-connector-python**: MySQL database driver

Architecture & Design

MVC Pattern Architecture




HTTP Request/Response
FASTAPI SERVER
main.py
- Application initialization
- Route registration
- Static file mounting
controller.py (Router)
- Route definitions
- Request handling
- Template rendering
- Form processing
schema.py (Models)
- Patient model
- Doctor model
- Field definitions & validation
database.py (Data Layer)
- Database connection
- Session management
- Table creation
SQL Queries
MYSQL DATABASE
hospital (Database)
doctor (Table)
id (INT, PRIMARY KEY)
name (VARCHAR)
specialty (VARCHAR)
phone (VARCHAR)
patient (Table)
id (INT, PRIMARY KEY)
name (VARCHAR)
age (INT)
gender (VARCHAR)

Design Patterns Used

1. Dependency Injection

- FastAPI's `Depends()` function manages database sessions automatically
- Decouples route handlers from session management
- Example: `session: Session = Depends(get_session)`

2. Repository Pattern

- Database operations encapsulated in `database.py`
- Clean separation between data access and business logic

3. Template Pattern

- Jinja2 templates provide consistent UI across pages
- Template inheritance for code reuse

4. ORM Pattern

- SQLModel maps database tables to Python classes
- Type-safe database operations with automatic validation

Database Schema

Doctor Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Unique doctor identifier
name	VARCHAR(20)	INDEX, NOT NULL	Doctor's full name
specialty	VARCHAR(30)	NOT NULL	Medical specialty (Cardiology, etc.)
phone	VARCHAR(10)	NOT NULL	Contact phone number

Table 1: Doctor Table Schema

Sample Data:

- Dr. Anil | Cardiology | 9876543210
- Dr. Meera | Gynecology | 9123456789
- Dr. Joseph | Orthopedics | 9988776655
- Dr. Ravi | Neurology | 9090969090

Patient Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO INCREMENT	Unique patient identifier
name	VARCHAR(30)	INDEX, NOT NULL	Patient's full name
age	INTEGER	NOT NULL	Patient's age in years
gender	VARCHAR(8)	NOT NULL	Gender (Male/Female)
phone	VARCHAR(10)	NOT NULL	Contact phone number
place	VARCHAR(20)	NOT NULL	City/Location
admission_date	DATE	NOT NULL	Date of hospital admission
doctor_name	VARCHAR(20)	NOT NULL	Assigned doctor's name
status	VARCHAR(20)	DEFAULT: 'Active'	Treatment status

Table 2: Patient Table Schema

Status Values: Admitted, Under Treatment, Discharged, Critical, Active

---

Code Structure & Components

1. main.py - Application Entry Point

Purpose: Initializes FastAPI application, configures startup events, mounts static files, and runs the server.

Key Components:

```
from fastapi import FastAPI
from database import create_db_and_tables
from controller import router
from fastapi.staticfiles import StaticFiles
import uvicorn

app = FastAPI()
```

```
@app.on_event("startup")
def on_startup():
create_db_and_tables()

app.mount("/static", StaticFiles(directory="static"), name="static")
app.include_router(router)

if __name__=="__main__":
uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

Execution Flow:

1. **FastAPI Initialization**: Creates FastAPI application instance
2. **Startup Event**: On server startup, `create_db_and_tables()` executes:
   - Connects to MySQL database
   - Creates tables if they don't exist
   - Prints confirmation message
3. **Static Files Mounting**: Maps `/static/` URL path to `static/` directory
   - Serves HTML templates from `static/home.html`, `static/form.html`, etc.
4. **Router Registration**: Includes all routes from `controller.py`
5. **Server Launch**: Starts Uvicorn development server on `127.0.0.1:8000`

---

2. database.py - Database Configuration

Purpose: Manages MySQL connection, creates tables on startup, and provides session management for database operations.

Key Components:

```
from sqlmodel import SQLModel, create_engine, Session
from typing import Generator
import os
from dotenv import load_dotenv

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL, echo=True)

def create_db_and_tables():
SQLModel.metadata.create_all(engine)
print("MySQL Connected")

def get_session() -> Generator[Session, None, None]:
with Session(engine) as session:
yield session
```

Key Features:

Environment Configuration:

- Uses `python-dotenv` to load `DATABASE_URL` from `.env` file
- Example `.env`: `DATABASE_URL=mysql+pymysql://root:password@localhost/hospital`

Database Engine:

- `create_engine()` creates SQLAlchemy engine with MySQL dialect
- `echo=True` logs all SQL queries to console (useful for debugging)

Auto Table Creation:

- `SQLModel.metadata.create_all(engine)` creates all tables defined in models
- Runs on application startup via `@app.on_event("startup")`
- Idempotent: safe to run multiple times

Session Management (Dependency Injection):

- `get_session()` is a generator function that yields database sessions
- Used with `Depends(get_session)` in route handlers
- Ensures sessions are properly created and closed
- Thread-safe for concurrent requests

---

3. schema.py - Data Models

Purpose: Defines Patient and Doctor models with field constraints and validation rules.

```
from sqlmodel import SQLModel, Field
from typing import Optional
from datetime import date

# PATIENT TABLE

class Patient(SQLModel, table=True):
id: Optional[int] = Field(default=None, primary_key=True)
name: str = Field(index=True, max_length=30)
age: int
gender: str = Field(max_length=8)
phone: str = Field(max_length=10)
place: str = Field(max_length=20)
admission_date: date
doctor_name: str = Field(max_length=20)
status: str = Field(default="Active")

# DOCTOR TABLE

class Doctor(SQLModel, table=True):
id: Optional[int] = Field(default=None, primary_key=True)
name: str = Field(index=True, max_length=20)
specialty: str = Field(max_length=30)
phone: str = Field(max_length=10)
```

Key Features:

SQLModel Integration:

- `table=True`: Makes class a database table
- Inherits from `SQLModel`: Provides both SQLAlchemy ORM and Pydantic validation

Field Constraints:

- primary_key=True: Sets ID as primary key with auto-increment
- index=True: Creates database index on name for faster queries
- max_length: Enforces string field length limits
- default=None: Optional field (nullable)
- default="Active": Sets default status value

Type Hints:

- Optional[int]: ID is optional (auto-generated on insert)
- str, int, date: Standard Python types with automatic validation
- Pydantic validates data before database insertion

Patient Model Fields:

- id: Auto-generated primary key
- name: Patient's full name (indexed for fast lookup)
- age: Age in years (integer)
- gender: Gender classification (Male/Female)
- phone: Contact number (10 digits)
- place: City/residence location
- admission_date: Date of hospital admission
- doctor_name: Name of assigned doctor
- status: Treatment status (default: Active)

Doctor Model Fields:

- id: Auto-generated primary key
- name: Doctor's full name (indexed)
- specialty: Medical specialty
- phone: Contact number

---

4. controller.py - Route Handlers

Purpose: Defines all HTTP routes, handles requests, manages database queries, and renders templates.

Route Overview:

```
from fastapi import APIRouter, Request, Form, Depends
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from sqlmodel import Session, select
from database import get_session
from schema import Patient, Doctor
from datetime import date

router = APIRouter()
templates = Jinja2Templates(directory="static")

Route 1: Home Page
@router.get("/", name="home", response_class=HTMLResponse)
```

```
def home(request: Request):
return templates.TemplateResponse("home.html", {"request": request})
```

- **Method**: GET
- **URL**: http://localhost:8000/
- **Purpose**: Serves home page
- **Template**: static/home.html

Route 2: Patient Form

```
@router.get("/Form", name="create_form", response_class=HTMLResponse)
def get_form(request: Request):
return templates.TemplateResponse("form.html", {"request": request})
```

- **Method**: GET
- **URL**: http://localhost:8000/Form
- **Purpose**: Displays form to add new patient
- **Template**: static/form.html

Route 3: Dashboard

```
@router.get("/Dashboard", name="dash_board", response_class=HTMLResponse)
def details(request: Request, session: Session = Depends(get_session)):
patients_count = len(session.exec(select(Patient)).all())
doctors_count = len(session.exec(select(Doctor)).all())
return templates.TemplateResponse("dashboard.html", {
"request": request,
"patients": patients_count,
"doctors": doctors_count
})
```

- **Method**: GET
- **URL**: http://localhost:8000/Dashboard
- **Purpose**: Shows statistics and overview
- **Database Operations**:
  - session.exec(select(Patient)).all(): Fetches all patients from database
  - session.exec(select(Doctor)).all(): Fetches all doctors
  - Counts records: len(...)
- **Template**: static/dashboard.html
- **Data Passed**: Patient count and doctor count

Route 4: View Patients

```
@router.get("/Patients", name="patient_details", response_class=HTMLResponse)
def get_patients(request: Request, session: Session = Depends(get_session)):
patients = session.exec(select(Patient)).all()
return templates.TemplateResponse("patients.html", {
"request": request,
"patients": patients
})
```

- **Method**: GET
- **URL**: http://localhost:8000/Patients
- **Purpose**: Displays all patients in a table
- **Database**: Retrieves all patient records

- **Template**: static/patients.html

Route 5: View Doctors

```
@router.get("/Doctors", name="doctor_details", response_class=HTMLResponse)
def get_doctors(request: Request, session: Session = Depends(get_session)):
doctors = session.exec(select(Doctor)).all()
return templates.TemplateResponse("doctors.html", {
"request": request,
"doctors": doctors
})
```

- **Method**: GET
- **URL**: http://localhost:8000/Doctors
- **Purpose**: Displays all doctors in a table
- **Database**: Retrieves all doctor records
- **Template**: static/doctors.html

Route 6: Create Patient (Form Submission)

```
@router.post("/patients/create", response_class=HTMLResponse)
def create_patient_form_post(
request: Request,
name: str = Form(...),
age: int = Form(...),
gender: str = Form(...),
phone: str = Form(...),
place: str = Form(...),
admission_date: date = Form(...),
doctor_name: str = Form(...),
status: str = Form(...),
session: Session = Depends(get_session)
):
db_patient = Patient(
name=name,
age=age,
gender=gender,
phone=phone,
place=place,
admission_date=admission_date,
doctor_name=doctor_name,
status=status
)
session.add(db_patient)
session.commit()
session.refresh(db_patient)
patients = session.exec(select(Patient)).all()
return templates.TemplateResponse("patients.html", {
"request": request,
"patients": patients
})
```

- **Method**: POST
- **URL**: http://localhost:8000/patients/create

- **Purpose**: Processes form submission and creates new patient
- **Form Parameters**: All patient fields (name, age, gender, phone, place, admission_date, doctor_name, status)
- **Database Operations**:
    1. Create Patient object with form data
    2. session.add(): Add to session
    3. session.commit(): Save to database
    4. session.refresh(): Get updated object with generated ID
    5. Fetch all patients for display
- **Response**: Redirects to patients list view showing all patients including newly created one

---

API Endpoints & Routes

Method	Endpoint	Purpose	Template	Parameters
GET	/	Home page	home.html	None
GET	/Form	Patient form	form.html	None
GET	/Dashboard	Statistics dashboard	dashboard.html	None
GET	/Patients	List all patients	patients.html	None
GET	/Doctors	List all doctors	doctors.html	None
POST	/patients/create	Add new patient	patients.html	Form data

Table 3: API Endpoints and Routes

Feature Walkthrough

Feature 1: Home Page

- **User Action**: User navigates to http://localhost:8000/
- **Backend**: home() route renders home.html
- **Frontend**: Displays welcome page with navigation menu
- **Navigation**: Links to Form, Dashboard, Patients, and Doctors pages

Feature 2: Add New Patient

- **User Action**: Clicks "Add Patient" or visits /Form
- **Frontend**: Form with fields:
    - Patient Name (text input)
    - Age (number input)
    - Gender (dropdown: Male/Female)

    - Phone (text input)
    - Place (text input)
    - Admission Date (date picker)
    - Doctor Name (text input)
    - Status (dropdown: Admitted, Under Treatment, Discharged, Critical)
- **Form Submission**: POST to `/patients/create`
- **Backend**:
    1. Receives form data
    2. Validates using Pydantic (via FastAPI)
    3. Creates Patient object
    4. Inserts into database
    5. Commits transaction
    6. Refreshes to get generated ID
    7. Fetches all patients for updated list
- **Frontend**: Displays patients table with new patient included

Feature 3: View All Patients

- **User Action**: Visits `/Patients`
- **Backend**:
    1. Queries database: `select(Patient).all()`
    2. Returns all patient records with all fields
- **Frontend**: Displays table with columns:
    - ID, Name, Age, Gender, Phone, Place, Admission Date, Doctor Name, Status
- **Data Displayed**: Sample patients like "Rahul Kumar", "Anjali Nair", "Priya Sharma", etc.

Feature 4: View All Doctors

- **User Action**: Visits `/Doctors`
- **Backend**:
    1. Queries database: `select(Doctor).all()`
    2. Returns all doctor records
- **Frontend**: Displays table with columns:
    - ID, Name, Specialty, Phone
- **Data Displayed**: Doctors like "Dr. Anil (Cardiology)", "Dr. Meera (Gynecology)", etc.

Feature 5: Dashboard

- **User Action**: Visits `/Dashboard`
- **Backend**:
    1. Counts all patients: `len(session.exec(select(Patient)).all())`
    2. Counts all doctors: `len(session.exec(select(Doctor)).all())`
    3. Returns counts with request object
- **Frontend**: Displays:
    - Total Patients count (e.g., 22)
    - Total Doctors count (e.g., 10)
    - Visual cards or statistics
    - Quick navigation links

Installation & Setup

Prerequisites

- Python 3.9 or higher
- MySQL Server installed and running
- Git (optional, for version control)
- Virtual environment (recommended)

Step 1: Clone or Create Project

mkdir hospital-management
cd hospital-management

Step 2: Create Virtual Environment

Windows

python -m venv venv
venv\Scripts\activate

Linux/Mac

python3 -m venv venv
source venv/bin/activate

Step 3: Install Dependencies

pip install fastapi
pip install uvicorn
pip install sqlmodel
pip install jinja2
pip install python-dotenv
pip install mysql-connector-python

Or create `requirements.txt`:
fastapi0.104.1
uvicorn0.24.0
sqlmodel0.0.14
jinja23.1.2
python-dotenv1.0.0
mysql-connector-python8.2.0

Then install: `pip install -r requirements.txt`

Step 4: Create Database

-- Open MySQL and create database
CREATE DATABASE hospital;
USE hospital;

-- Tables will be auto-created by SQLModel

Step 5: Create .env File

In project root directory, create `.env`:
DATABASE_URL=mysql+pymysql://root:your_password@localhost:3306/hospital

Replace:

- `root` with your MySQL username
- `your_password` with your MySQL password
- `localhost` with your MySQL host
- `3306` with your MySQL port (if different)

Step 6: Create Directory Structure

hospital-management/
├── main.py
├── controller.py
├── schema.py
├── database.py
├── .env
├── requirements.txt
└── static/
├── home.html
├── form.html
├── dashboard.html
├── patients.html
└── doctors.html

Step 7: Run Application

python main.py

Or directly with Uvicorn:
uvicorn main:app --host 127.0.0.1 --port 8000 --reload

Output:
INFO: Application startup complete
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

Step 8: Access Application

Open browser and navigate to:

- Home: http://localhost:8000/
- Add Patient: http://localhost:8000/Form
- Dashboard: http://localhost:8000/Dashboard
- View Patients: http://localhost:8000/Patients
- View Doctors: http://localhost:8000/Doctors

Usage Guide

Adding a Patient

1. Navigate to http://localhost:8000/Form
2. Fill in form fields:
    - **Patient Name**: Enter full name
    - **Age**: Enter age in years
    - **Gender**: Select Male or Female
    - **Phone**: Enter 10-digit phone number
    - **Place**: Enter city/location
    - **Admission Date**: Pick from calendar or enter manually
    - **Doctor Name**: Enter assigned doctor's name
    - **Status**: Select from Admitted, Under Treatment, Discharged, or Critical
3. Click "Submit" or "Add Patient"
4. Form submits to /patients/create endpoint
5. Patient record created in database
6. Redirected to Patients page showing all patients including new one

Viewing Patients

1. Navigate to http://localhost:8000/Patients
2. See table with all patients and their details
3. Columns show: ID, Name, Age, Gender, Phone, Place, Admission Date, Doctor Name, Status

Viewing Doctors

1. Navigate to http://localhost:8000/Doctors
2. See table with all doctors and their details
3. Columns show: ID, Name, Specialty, Phone

Checking Dashboard

1. Navigate to http://localhost:8000/Dashboard
2. See total count of patients and doctors
3. View summary statistics and quick links

Key Implementation Details

Dependency Injection Pattern

FastAPI's dependency injection simplifies database session management:

def get_session() -> Generator[Session, None, None]:
with Session(engine) as session:
yield session

@router.get("/Patients")
def get_patients(request: Request, session: Session = Depends(get_session)):
patients = session.exec(select(Patient)).all()
# session automatically closed after route completes

Benefits:

- Automatic session creation and cleanup
- Thread-safe for concurrent requests
- Testable (can inject mock sessions)
- Decoupled from route logic

Form Handling with FastAPI

FastAPI automatically handles form data parsing:

@router.post("/patients/create")
def create_patient_form_post(
request: Request,
name: str = Form(...), # Required field
age: int = Form(...), # Type validated and converted
admission_date: date = Form(...) # Date parsing automatic
):
Form data validated before function call

Features:

- Automatic type conversion (`str`, `int`, `date`)
- Required field validation (`Form(...)`)
- Error handling and validation messages

SQLModel ORM Operations

SELECT - Fetch all records

patients = session.exec(select(Patient)).all()

SELECT - Fetch with condition

patient = session.exec(select(Patient)).first()

INSERT - Create new record

db_patient = Patient(name="John", age=30, ...)
session.add(db_patient)
session.commit()

UPDATE - Modify record (implicit via commit)

session.refresh(db_patient)

DELETE - Remove record (if implemented)

session.delete(db_patient)
session.commit()

Jinja2 Template Rendering

from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="static")

@router.get("/Patients")
def get_patients(request: Request, session: Session = Depends(get_session)):
patients = session.exec(select(Patient)).all()
return templates.TemplateResponse("patients.html", {
"request": request, # Required for Jinja2
"patients": patients # Data for template
})

In Template (patients.html):
{% for patient in patients %}
{{ patient.id }} {{ patient.name }} {{ patient.age }}
{% endfor %}

Database Connection Flow

Application Start
↓
@app.on_event("startup") triggers
↓
create_db_and_tables() executes
↓
SQLModel.metadata.create_all(engine)

↓
Connects to MySQL via DATABASE_URL
↓
Creates tables from model definitions
↓
Prints "MySQL Connected"
↓
Application ready to accept requests

Request-Response Lifecycle

User Browser Request (GET /Patients)
↓
FastAPI Router matches route
↓
Dependencies resolved: Depends(get_session)
↓
Database session created
↓
Route handler executes: get_patients(request, session)
↓
Database query: session.exec(select(Patient)).all()
↓
Fetch all patient records from MySQL
↓
Create dict with request and patients data
↓
Jinja2 renders template with data
↓
HTML response generated
↓
Session closed (automatically)
↓
HTML sent to browser
↓
User sees rendered page

Common Scenarios & Solutions

Scenario 1: How to add a validation rule?

Modify `schema.py`:
from pydantic import field_validator

class Patient(SQLModel, table=True):
phone: str = Field(max_length=10)

```
@field_validator('phone')
@classmethod
```

```
    def validate_phone(cls, v):
        if not v.isdigit():
            raise ValueError('Phone must contain only digits')
        return v
```

Scenario 2: How to add search functionality?

Add new route to `controller.py`:

```
@router.get("/patients/search/{name}")
def search_patients(request: Request, name: str, session: Session = Depends(get_session)):
patients = session.exec(
select(Patient).where(Patient.name.contains(name))
).all()
return templates.TemplateResponse("patients.html", {
"request": request,
"patients": patients
})
```

Scenario 3: How to delete a patient?

Add new route to `controller.py`:

```
@router.delete("/patients/{patient_id}")
def delete_patient(patient_id: int, session: Session = Depends(get_session)):
patient = session.exec(
select(Patient).where(Patient.id == patient_id)
).first()
if patient:
session.delete(patient)
session.commit()
return {"message": "Patient deleted"}
```

Scenario 4: How to sort patients by name?

Modify route in `controller.py`:

```
@router.get("/Patients")
def get_patients(request: Request, session: Session = Depends(get_session)):
patients = session.exec(
select(Patient).order_by(Patient.name)
).all()
return templates.TemplateResponse("patients.html", {
"request": request,
"patients": patients
})
```

Performance Optimization Tips

1. Database Indexing

- `name` field in both Patient and Doctor tables already indexed
- Speeds up `where` queries: `session.exec(select(Patient).where(Patient.name == ...))`

2. Pagination

For large patient lists, implement pagination:
@router.get("/Patients")
def get_patients(request: Request, skip: int = 0, limit: int = 10,
session: Session = Depends(get_session)):
patients = session.exec(
select(Patient).offset(skip).limit(limit)
).all()

3. Query Optimization

- Use `first()` instead of `all()` when fetching single record
- Use `select()` with specific columns instead of `*`

4. Caching

- Consider caching doctor list (infrequently changed)
- Use Redis for high-traffic applications

5. Connection Pooling

Already handled by SQLAlchemy engine with default pool settings.

Security Considerations

1. SQL Injection Prevention

✓ **Already protected** by SQLModel/SQLAlchemy ORM

- Uses parameterized queries
- Never concatenate user input into SQL

SAFE - SQLModel handles escaping

patients = session.exec(select(Patient).where(Patient.name == user_input)).all()

NEVER DO THIS

query = f"SELECT * FROM patient WHERE name = '{user_input}'" # Vulnerable!

2. Password Security

Currently no authentication. For production, add:
from fastapi.security import HTTPBasic, HTTPBasicCredentials
security = HTTPBasic()

@router.get("/protected")
def protected_route(request: Request, credentials: HTTPBasicCredentials =
Depends(security)):
Verify credentials
pass

3. Input Validation

✓ **Already implemented** via Pydantic in schema

- Type validation
- Length constraints
- Required field enforcement

4. CORS Protection

For frontend from different domain:
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
CORSMiddleware,
allow_origins=["http://localhost:3000"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)

5. HTTPS

Use HTTPS in production (deploy with `--ssl-keyfile` and `--ssl-certfile`).

Conclusion

The Hospital Management System demonstrates a complete, production-ready full-stack application combining:

✓ **Modern Backend Framework**: FastAPI with automatic documentation and validation
✓ **Robust Database**: MySQL with proper schema design and ORM operations
✓ **User-Friendly Frontend**: Jinja2 templates for server-side rendering
✓ **Clean Architecture**: Separation of concerns with MVC pattern

✓ **Type Safety**: Python type hints and Pydantic validation
✓ **Scalability**: Dependency injection and proper session management

This project serves as an excellent foundation for learning:

- Full-stack web development
- Database design and relationships
- RESTful API principles
- Template rendering
- Form handling and validation
- Authentication and authorization (future enhancement)

Future Enhancement Opportunities:

- Add patient update/delete functionality
- Implement doctor management (add/update/delete)
- Add authentication and user roles
- Implement appointment scheduling
- Add medical records/prescription management
- Email notifications for patients
- Generate reports and statistics
- Mobile-responsive design improvements
- API testing with pytest
- Docker containerization for deployment

References

[1] FastAPI Official Documentation. (2024). https://fastapi.tiangolo.com/

[2] SQLModel Documentation. (2024). https://sqlmodel.tiangolo.com/

[3] SQLAlchemy ORM Tutorial. (2024). https://docs.sqlalchemy.org/en/20/orm/

[4] Jinja2 Template Documentation. (2024). https://jinja.palletsprojects.com/

[5] MySQL Database Manual. (2024). https://dev.mysql.com/doc/

[6] Uvicorn ASGI Server. (2024). https://www.uvicorn.org/

[7] Pydantic Data Validation. (2024). https://docs.pydantic.dev/

[8] FastAPI Form Data Handling. (2024). https://fastapi.tiangolo.com/tutorial/request-forms/